Lecture_10_Visible_Surface_Detection_and_Z_Buffer

CSTE 3201 Computer Graphics

Lecture 10: Visible Surface Detection and Z-Buffer

> **Previous lecture connection:** Lecture 9 explained how a 3D scene is viewed through a camera, clipped against the view volume, projected, and mapped to the viewport. After that, another important question remains: if multiple projected surfaces cover the same screen pixel, **which one should be visible?** This lecture answers that question.

Learning Outcomes

After completing this lecture, students should be able to:

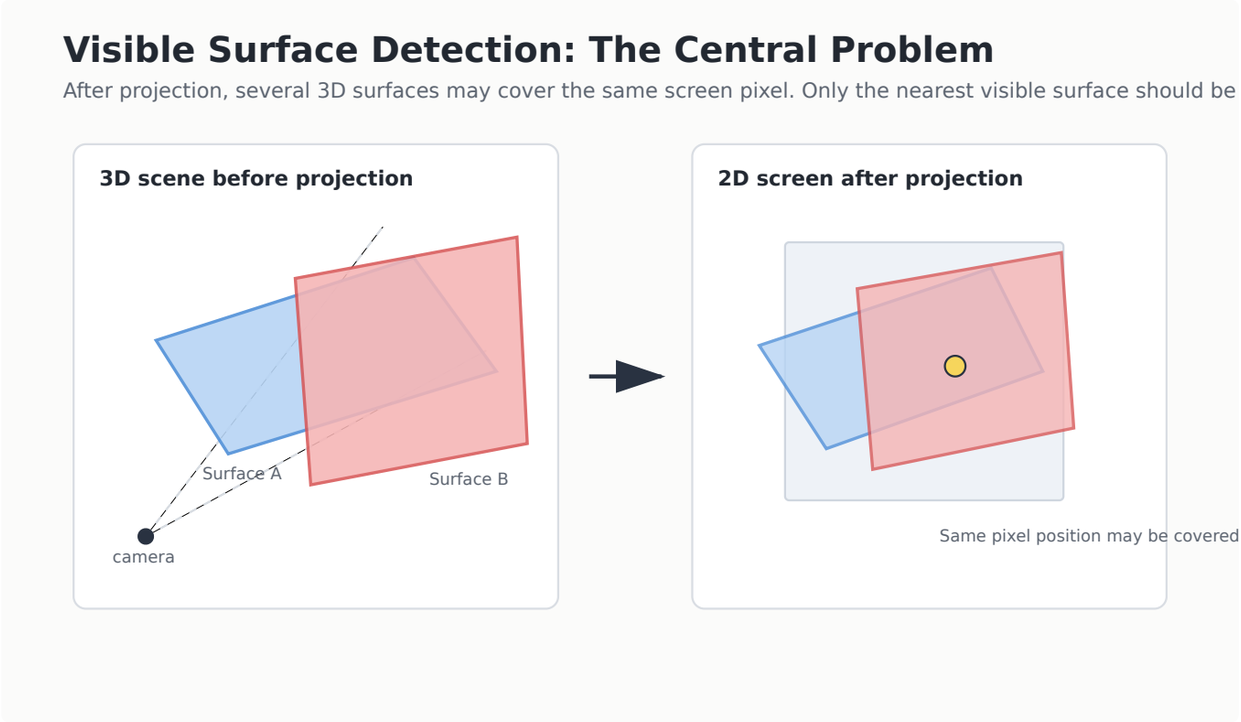
1. Explain the visible-surface detection problem in 3D graphics.
2. Distinguish between clipping and hidden-surface removal.
3. Explain the basic idea of the z-buffer algorithm.
4. Apply a simple depth comparison at a pixel.
5. Describe how the color buffer and depth buffer work together.
6. Explain why depth interpolation is needed during rasterization.
7. Describe back-face culling and its role in rendering.
8. Identify common visibility problems such as z-fighting.
9. Compare image-space and object-space visibility methods.
10. Recognize practical applications of visible-surface detection.

---

1. The Visibility Problem

When a 3D scene is projected onto a 2D screen, many different surfaces may overlap in screen space. A wall, a chair, and a table may all project onto the same region of the image. However, for each pixel, the viewer should see only the surface that is nearest to the camera.

This is the **visible-surface detection problem**, also called the **hidden-surface removal problem**.



A simple way to state the problem is:

> Given many surfaces projected onto the screen, determine which surface is visible at each pixel.

This problem is different from projection. Projection only tells where a point appears on the screen. Visibility decides whether that point should actually be seen.

---

2. Why Projection Alone Is Not Enough

Suppose two triangles are projected onto the same screen pixel. Projection gives the same pixel coordinate for both triangles. But the screen cannot show two unrelated colors at exactly the same pixel at the same time. Therefore, the renderer must choose one.

Usually, the chosen surface is the one with the smallest distance from the camera along the viewing direction.

For example:

- A red triangle covers pixel `(100, 80)` with depth `0.60`.
- A blue triangle covers the same pixel with depth `0.35`.
- If smaller depth means nearer, the blue triangle should be visible.

This is why depth information is needed even after the 3D scene has been projected to 2D.

---

3. Clipping vs Hidden-Surface Removal

Students often confuse **clipping** and **hidden-surface removal**. They solve different problems.

3.1 Clipping

Clipping removes geometry outside the viewing volume.

Example: If a triangle is completely behind the camera or outside the left clipping plane, it should not be processed further.

3.2 Hidden-Surface Removal

Hidden-surface removal handles geometry that is inside the view volume but blocked by other geometry.

Example: A cube may be fully inside the view volume, but its back face is hidden by its front face.

So:

> Clipping asks: **Is the object inside the camera's view?**
> Visibility asks: **Among visible-region objects, which surface is nearest at this pixel?**

---

4. Two Broad Approaches

Visible-surface detection methods can be broadly divided into two categories.

4.1 Object-Space Methods

Object-space methods compare objects or surfaces with one another before final rasterization.

Examples:

- depth sorting,
- binary space partitioning,
- analytic polygon comparison.

These methods reason about geometry directly.

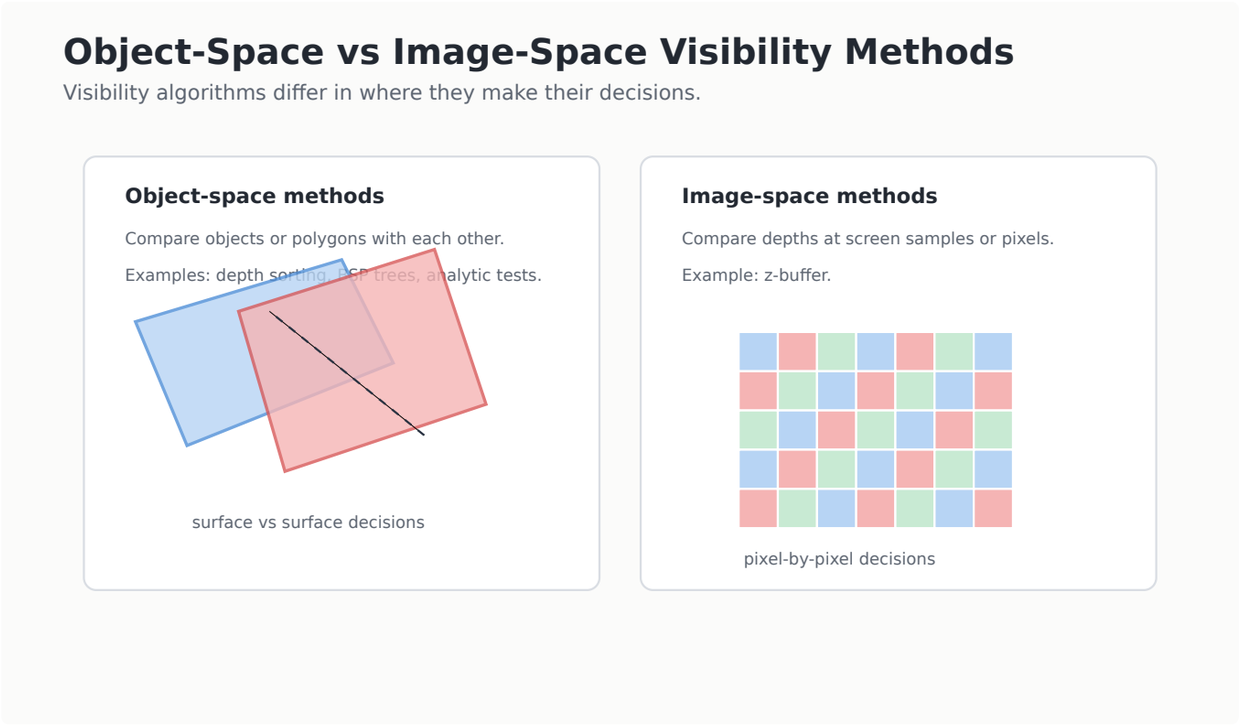
4.2 Image-Space Methods

Image-space methods decide visibility at the pixel or sample level.

Examples:

- z-buffer,
- A-buffer,
- ray casting per pixel.

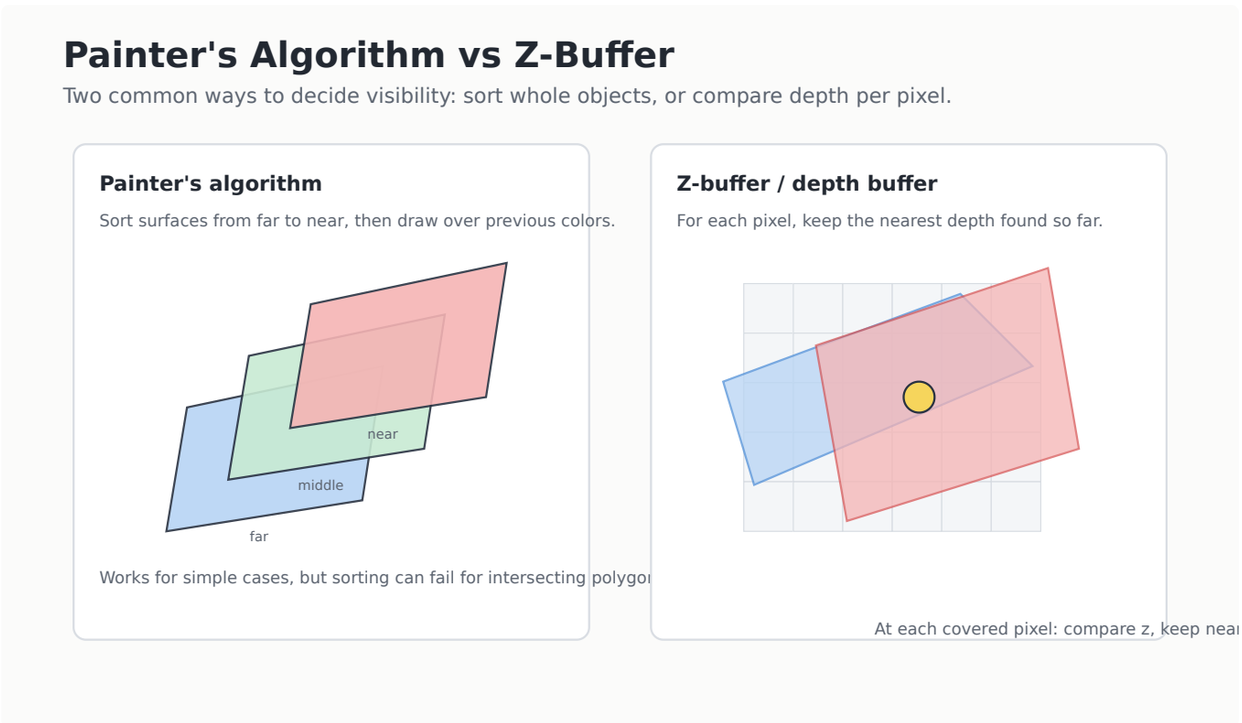
The z-buffer is the most common introductory method because it is simple and practical.



5. Painter's Algorithm

One intuitive method is the **painter's algorithm**. The idea is similar to painting a picture:

1. Draw far surfaces first.
2. Draw nearer surfaces later.
3. Nearer surfaces overwrite farther surfaces.



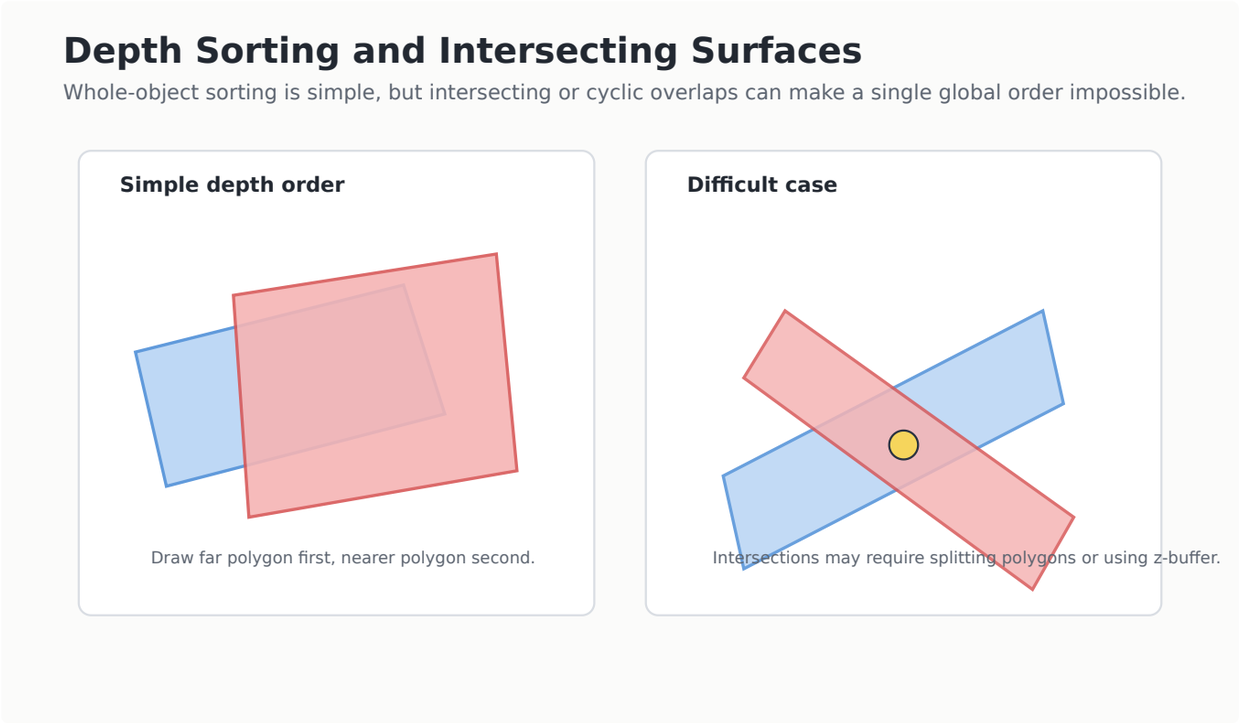
This can work well when objects can be sorted from back to front. However, it has serious limitations.

5.1 Problems with Painter's Algorithm

Painter's algorithm can fail when:

- polygons intersect each other,
- objects cyclically overlap,
- one polygon is partly in front and partly behind another,

- sorting is expensive or unstable.



To fix difficult cases, polygons may need to be split. This makes the method more complex.

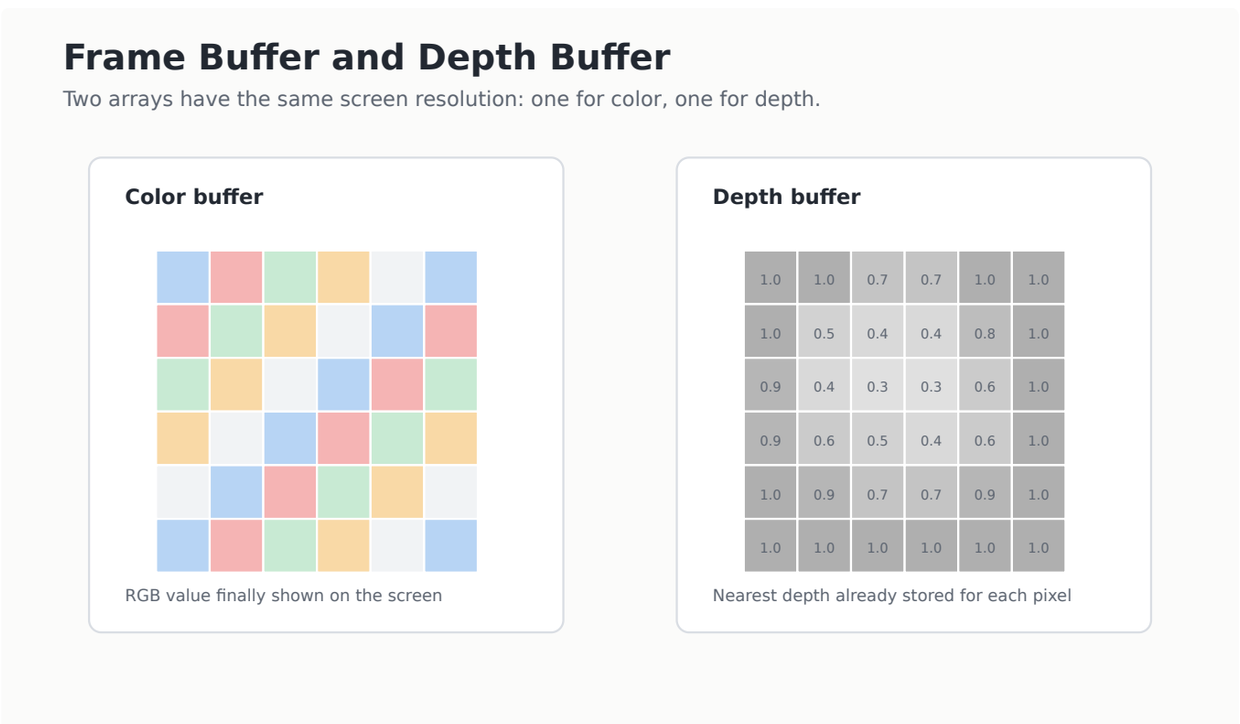
---

6. The Z-Buffer Idea

The **z-buffer algorithm**, also called the **depth-buffer algorithm**, solves visibility at the pixel level.

For every pixel, the renderer stores two things:

1. **Color value**: the color currently visible at that pixel.
2. **Depth value**: the depth of the currently visible surface at that pixel.



The z-buffer asks a very simple question whenever a new surface wants to color a pixel:

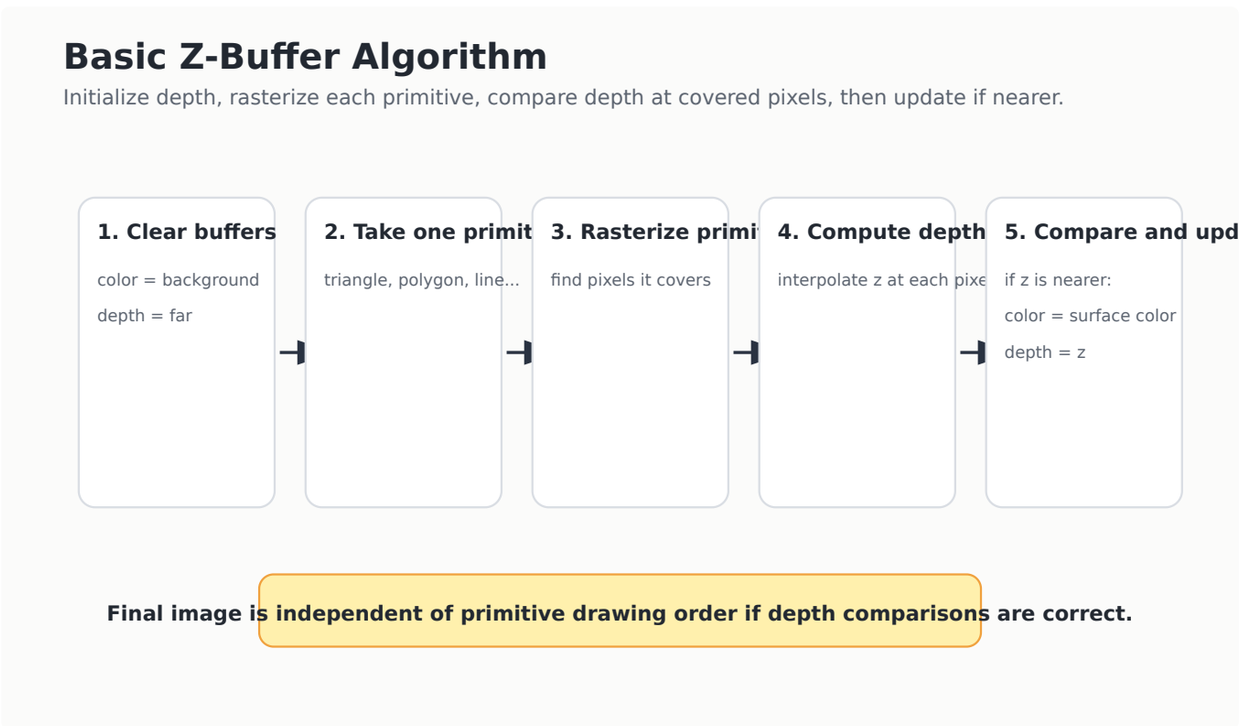
> Is this new surface closer than the surface already stored at this pixel?

If yes, update the pixel color and depth. If no, ignore the new surface for that pixel.

7. Basic Z-Buffer Algorithm

A simple version of the z-buffer algorithm is:

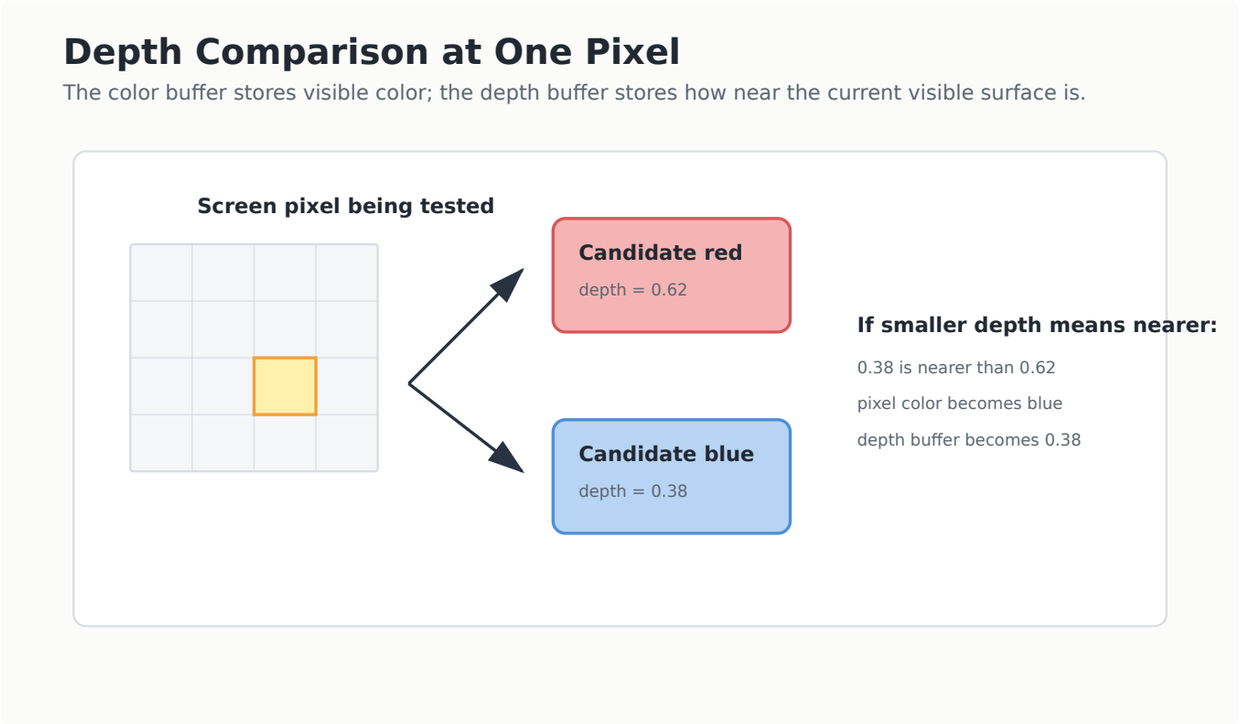
```
for every pixel (x, y):
    color_buffer[x, y] = background_color
    depth_buffer[x, y] = farthest_depth

for every polygon in the scene:
    rasterize the polygon
    for every pixel (x, y) covered by the polygon:
        compute polygon depth z at (x, y)
        if z is nearer than depth_buffer[x, y]:
            depth_buffer[x, y] = z
            color_buffer[x, y] = polygon_color
```



The exact comparison depends on the depth convention. In many normalized systems, smaller depth means nearer; in some conventions, larger depth may mean nearer. The important idea is consistent comparison.

8. A Single-Pixel Depth Comparison

Consider one screen pixel covered by two surfaces.



Suppose the current depth buffer value at the pixel is `0.62`, and a new fragment has depth `0.38`.

If smaller depth means nearer:

```
0.38 < 0.62
```

So the new fragment is closer. The renderer updates both buffers:

```
depth_buffer[x, y] = 0.38
color_buffer[x, y] = new_fragment_color
```

If the new fragment had depth `0.75`, it would be farther and would be rejected.

---

9. Why a Depth Buffer Is Needed for Every Pixel

A surface is not simply in front or behind another surface globally. It may be in front at one pixel and behind at another pixel.

This is especially true when:

- polygons intersect,
- surfaces are tilted,
- objects overlap partially,
- the viewpoint changes.

Therefore, depth is stored per pixel rather than per object.

The z-buffer does not need to sort all objects first. It can process polygons in almost any order, as long as the depth test is applied correctly.

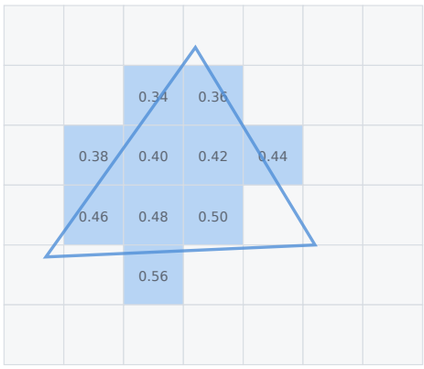
---

10. Depth Values During Rasterization

When a triangle is rasterized, it covers a set of pixels. Each covered pixel needs a depth value.

Example: Rasterized Triangle with Depth Values

A triangle covers only some pixels. Depth must be computed for each covered pixel.



Triangle vertices have depth values.

Interior pixel depths are obtained by interpolation.

Then each depth is compared with the existing value in the depth buffer at the same pixel.

The triangle's vertices have known depth values. Interior pixel depths are obtained by interpolation.

For a planar triangle, depth changes smoothly across the triangle. A simple conceptual form is:

```
z(x, y) = a x + b y + c
```

where `a`, `b`, and `c` are constants for that triangle after setup.

---

11. Depth Interpolation

Depth interpolation is necessary because different points on the same polygon may have different distances from the camera.

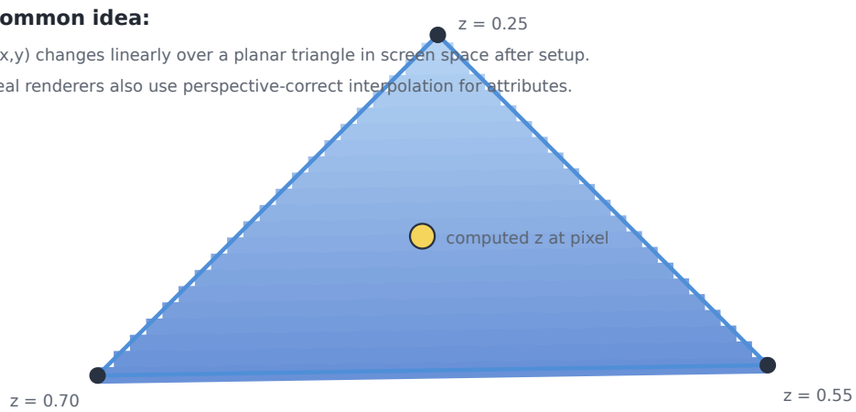
Depth Interpolation Across a Triangle

For a planar triangle, depth changes smoothly across the surface.

Common idea:

z(x,y) changes linearly over a planar triangle in screen space after setup.

Real renderers also use perspective-correct interpolation for attributes.



For example, a triangle may have vertex depths:

```
A: z = 0.25
B: z = 0.70
C: z = 0.55
```

A pixel inside the triangle may have a depth such as `0.43`, computed by interpolation.

In real graphics systems, depth and other attributes are handled carefully after perspective projection. Texture coordinates and colors often need **perspective-correct interpolation**, but the central idea remains: each covered pixel gets its own depth value.

---

12. Worked Example: Depth Buffer Update

Assume smaller depth means nearer. At pixel `(20, 15)`, the depth buffer currently stores:

```
depth_buffer[20, 15] = 0.55
color_buffer[20, 15] = green
```

Now a red triangle covers the same pixel with depth `0.42`.

Since:

```
0.42 < 0.55
```

red is nearer. So:

```
depth_buffer[20, 15] = 0.42
color_buffer[20, 15] = red
```

Later, a blue triangle covers the pixel with depth `0.60`.

Since:

```
0.60 > 0.42
```

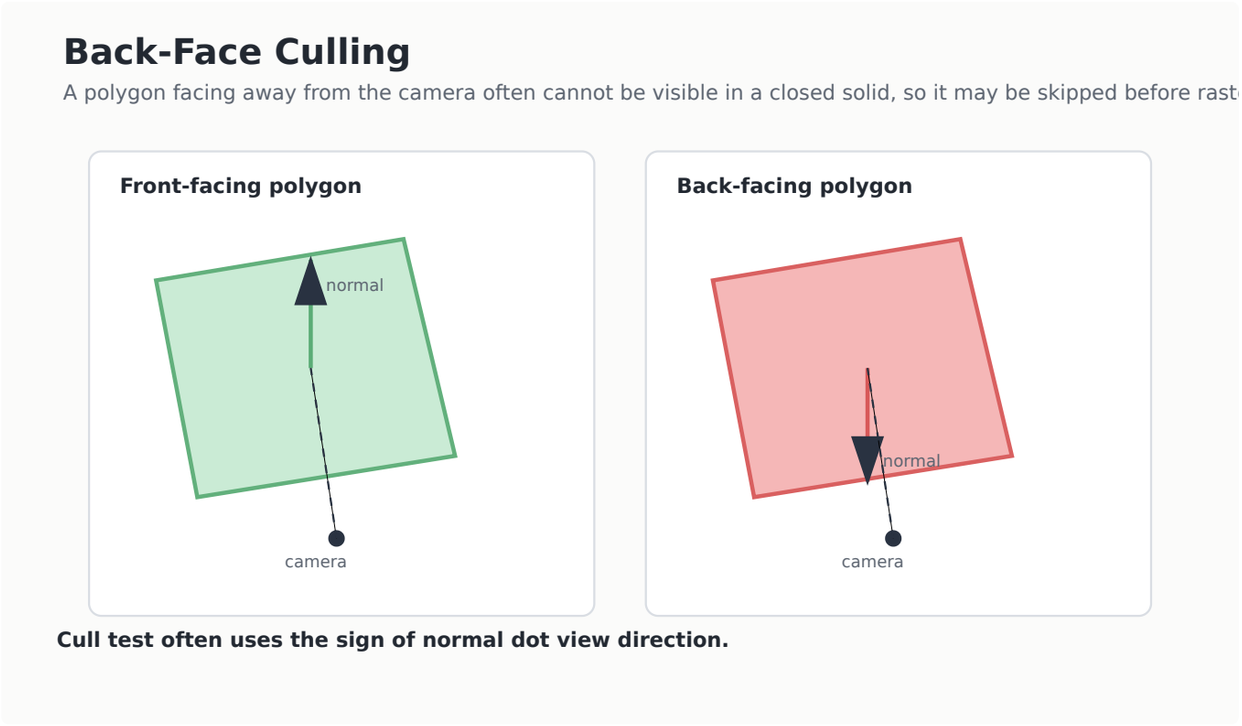
blue is farther, so it is rejected.

Final visible color at that pixel: **red**.

---

13. Back-Face Culling

Before using the z-buffer, many renderers remove surfaces that face away from the camera. This is called **back-face culling**.



For a closed solid object, a back-facing polygon usually cannot be seen because it is on the far side of the object.

Back-face culling is not the same as z-buffering:

- Back-face culling removes some polygons before rasterization.
- Z-buffering resolves visibility among the remaining projected fragments.

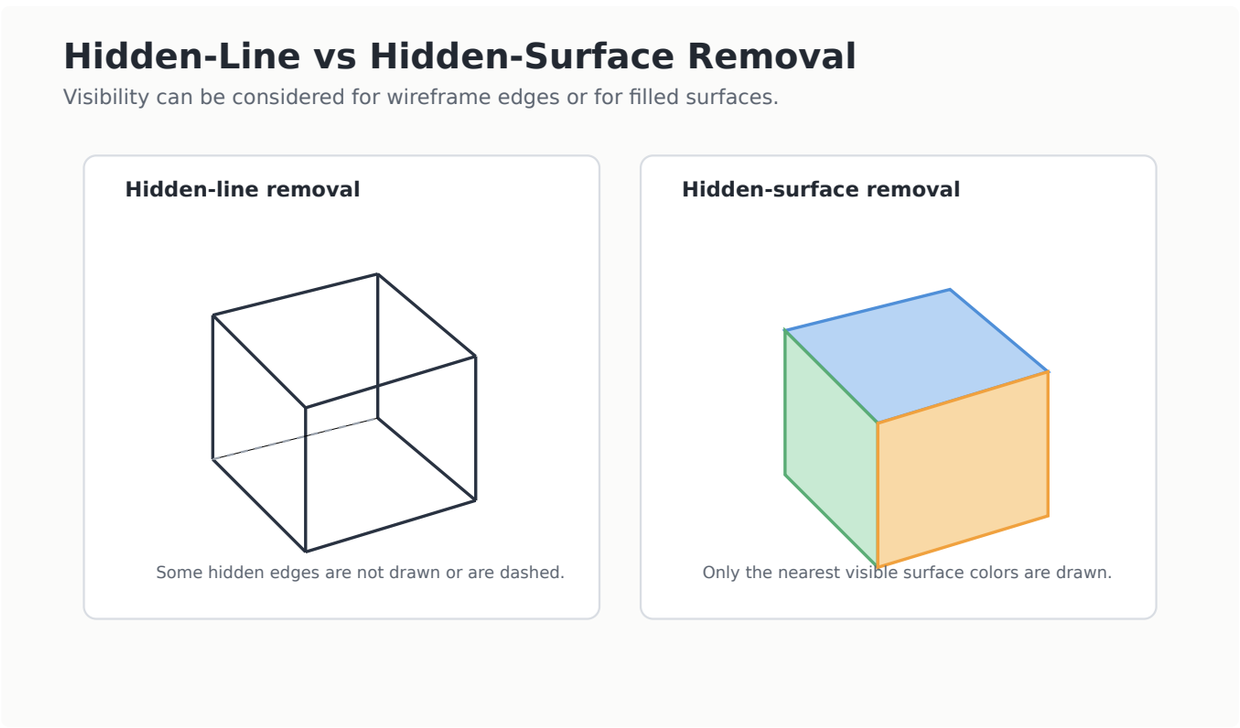
A common test uses the surface normal and the viewing direction. If the polygon normal points away from the viewer, the polygon can be skipped.

---

14. Hidden-Line and Hidden-Surface Removal

In wireframe drawings, the main problem is deciding which edges should be visible. This is called **hidden-line removal**.

In solid rendering, the main problem is deciding which filled surfaces should be visible. This is called **hidden-surface removal**.



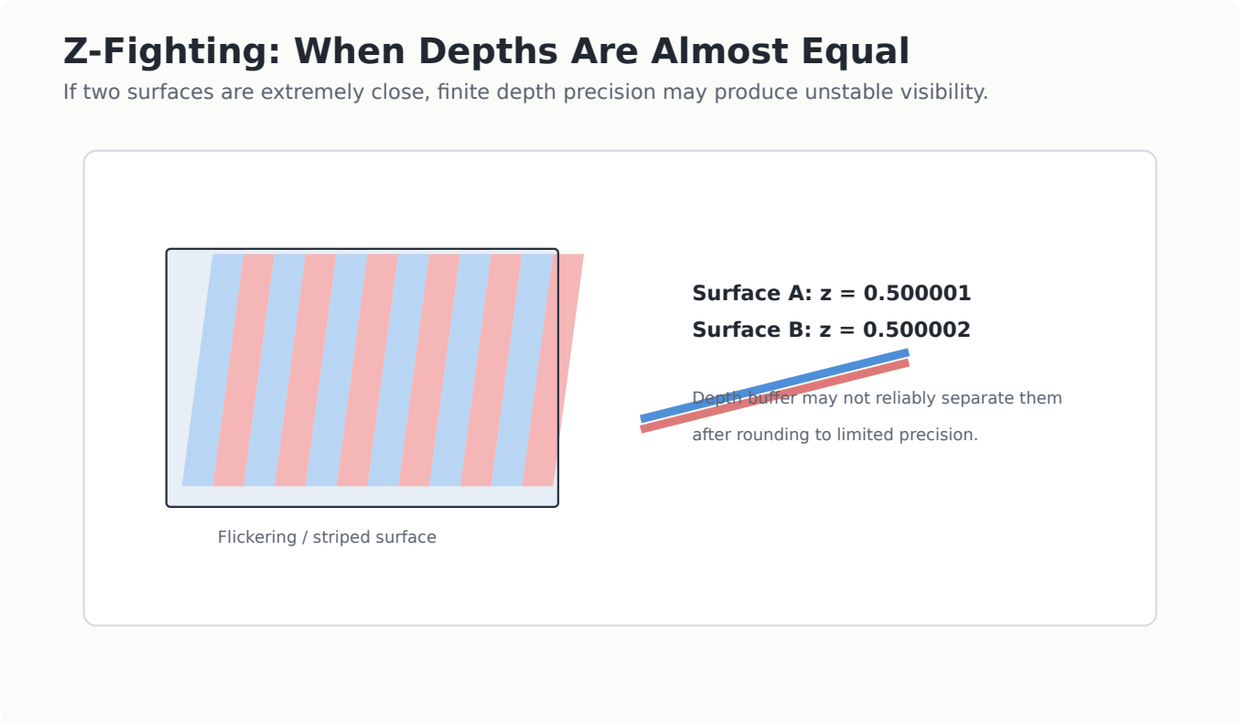
Hidden-line removal is important in engineering drawings and CAD systems. Hidden-surface removal is essential in shaded 3D rendering.

---

15. Z-Fighting and Depth Precision

The z-buffer has finite precision. Common depth buffers may use 16, 24, or 32 bits. If two surfaces are extremely close together, their stored depth values may become nearly identical.

This can cause **z-fighting**.



Z-fighting often appears as flickering or striped patterns as the camera moves.

Common causes include:

- duplicate surfaces,
- two polygons occupying almost the same plane,
- very large far-plane distance,
- very small near-plane distance,
- insufficient depth-buffer precision.

A practical fix is to avoid placing surfaces exactly on top of each other and to choose sensible near and far clipping planes.

---

16. Important Practical Notes

16.1 Drawing Order

With z-buffering, drawing order usually does not affect the final visible result for opaque surfaces. This is one reason the z-buffer is popular.

However, transparent surfaces need special handling. A transparent glass pane cannot always be rendered correctly by simple z-buffer replacement alone.

16.2 Memory Cost

A depth buffer requires extra memory. For a display of size `W x H`, the depth buffer needs one depth value per pixel.

Example:

```
1920 x 1080 = 2,073,600 pixels
```

If each depth value uses 24 bits, the depth buffer uses about:

```
2,073,600 x 24 bits = 49,766,400 bits
= 6,220,800 bytes
about 5.9 MiB
```

This is usually acceptable for modern graphics hardware.

16.3 Depth Convention

Different systems use different depth ranges, such as:

```
0 to 1
```

or

```
−1 to 1
```

Some systems define smaller depth as nearer, while others use a reversed depth convention. Always check the convention before applying a numerical comparison.

---

17. Common Mistakes

Mistake 1: Thinking Clipping Solves Visibility

Clipping removes geometry outside the view volume. It does not decide which of two overlapping visible surfaces is nearer.

Mistake 2: Thinking One Depth Value Per Object Is Enough

A tilted polygon may be near on one side and far on another. Depth must be considered at pixel level.

Mistake 3: Ignoring Depth Precision

Even if the z-buffer algorithm is conceptually correct, limited precision can cause visual artifacts.

Mistake 4: Using the Wrong Depth Comparison

If the system uses smaller depth as nearer, use `<`. If the system uses larger depth as nearer, use `>`. Mixing conventions gives incorrect visibility.

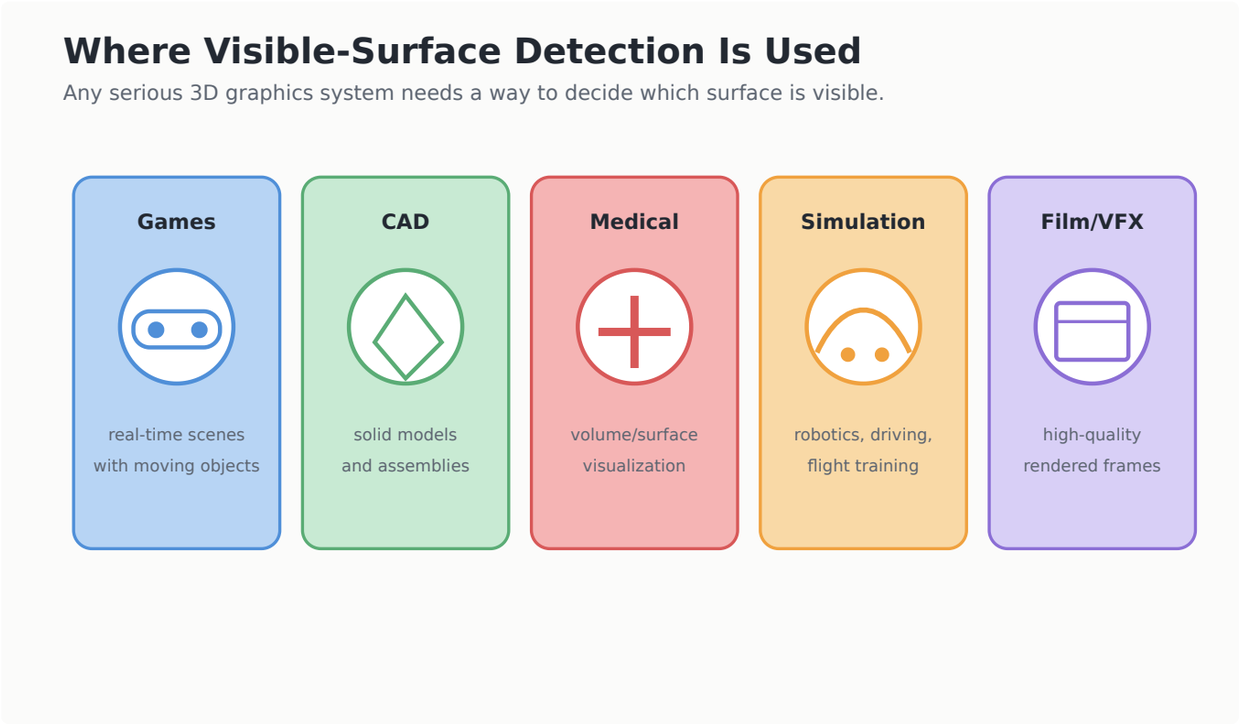
Mistake 5: Assuming Transparency Works Like Opaque Surfaces

Opaque surfaces work well with simple z-buffering. Transparent surfaces often need sorting, blending rules, or additional techniques.

---

18. Applications

Visible-surface detection is used in almost every 3D graphics application.



18.1 Video Games

Games use depth buffers continuously to render complex real-time worlds.

18.2 CAD and Engineering

CAD systems use hidden-line and hidden-surface removal to display solid objects clearly.

18.3 Medical Visualization

3D models of organs, bones, and scanned structures require correct visibility to be interpretable.

18.4 Simulation and Training

Flight simulators, driving simulators, and robotics simulators need correct occlusion to look realistic.

18.5 Film and Visual Effects

High-quality rendering systems solve visibility along with lighting, shading, and reflection.

---

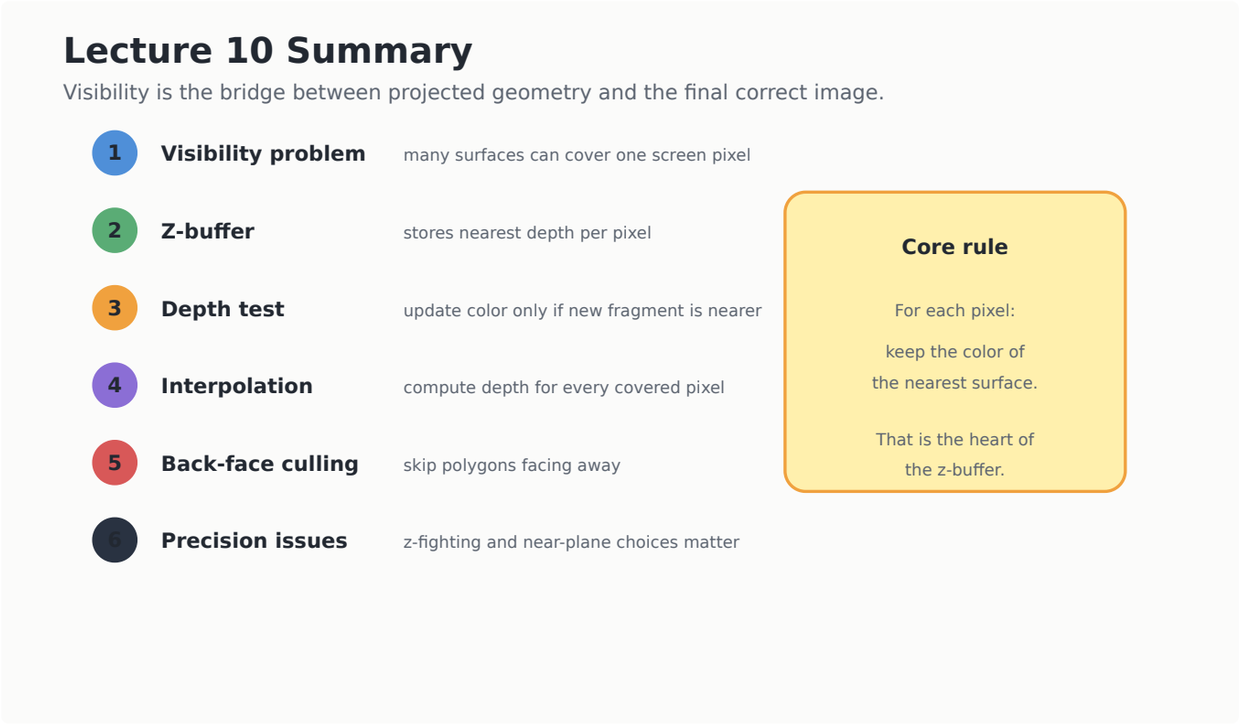
19. Trivia

- The z-buffer became popular because it is simple and maps well to hardware.
- The term **z-buffer** comes from the use of the z-coordinate as a depth measure.
- Some modern systems use **reversed-Z** depth buffers to improve precision.
- Hidden-surface removal was a major challenge in early 3D graphics because memory and computation were limited.

- A depth buffer stores geometry-related information, but it is arranged like an image: one value per pixel.

---

20. Lecture Summary



Key ideas:

- Projection maps 3D geometry to the screen, but visibility decides what is actually seen.
- Clipping removes objects outside the view volume; hidden-surface removal handles occlusion inside the view.
- The z-buffer stores the nearest depth found so far for each pixel.
- The color buffer stores the visible color for each pixel.
- During rasterization, each covered pixel receives an interpolated depth value.
- A fragment updates the pixel only if it passes the depth test.
- Back-face culling can reduce work before rasterization.
- Z-fighting occurs when surfaces have nearly equal depth values.
- Object-space methods compare surfaces geometrically; image-space methods compare pixels or samples.

---

21. Quick Concept Checks

1. Why is projection alone not enough to create a correct 3D image?
2. What is the difference between clipping and hidden-surface removal?
3. What two values are commonly stored for every pixel when z-buffering is used?
4. Why does a triangle need depth interpolation during rasterization?
5. What happens if a new fragment is farther than the current depth-buffer value?
6. Why can painter's algorithm fail for intersecting polygons?
7. What is z-fighting?
8. Why is back-face culling useful?

---

22. Exam-Style Questions

Short Questions

1. Define visible-surface detection.
2. What is the purpose of a depth buffer?
3. Explain the difference between a color buffer and a depth buffer.
4. What is back-face culling?
5. What is z-fighting?

Analytical Questions

1. At a pixel, the depth buffer stores `0.48`. A new fragment arrives with depth `0.35`. If smaller depth means nearer, should the pixel be updated? Explain.
2. Why is the z-buffer considered an image-space method?
3. A triangle has vertices at different depths. Why is it not enough to use only the average depth of the triangle?
4. Explain why transparent surfaces are more difficult than opaque surfaces for z-buffer rendering.
5. A student says, "If an object is inside the view frustum, it must be visible." Explain why this statement is incomplete.

---

23. Suggested Reading / Preparation for Next Lecture

Before the next lecture, review:

- polygon meshes,
- vertices, edges, and faces,
- parametric curves,
- wireframe representation,
- surface modeling.

The next lecture will move from **which surface is visible** to **how 3D objects and curved shapes are represented mathematically**.